

0928 Answer | 谁在运行、谁在等待、谁进入内核

原题 3 道 · 0926 的 scanf/内核态 · 能力账本

先沿事件发生顺序标注进程状态；再区分数据所在的地址区与处理该事件的执行主体。读懂尚非限时独立。

考场入口

题	先找信息	答案锚点
01	数值越小越先调度	<code>nice+cpuTime-waitTime</code> 是一种抗饥饿动态优先级
02	阻塞→键盘输入→中断→读缓冲→就绪→返回	②⑥④③①⑤；驱动负责③
03	声明位置与 <code>malloc</code> 返回的指向位置不同	PCB内核、main代码、ptr数据、length栈、字符串堆

01 | 2016-46：等待时间要让优先级数值下降

46. 某进程调度程序采用基于优先数（priority）的调度策略，即选择优先数最小的进程运行，进程创建时由用户指定一个 nice 作为静态优先数。为了动态调整优先数，引入运行时间 cpuTime 和等待时间 waitTime，初值均为 0。进程处于执行态时，cpuTime 定时加 1，且 waitTime 置 0；进程处于就绪态时，cpuTime 置 0，waitTime 定时加 1。请回答下列问题。

（1）若调度程序只将 nice 的值作为进程的优先数，即 $\text{priority} = \text{nice}$ ，则可能会出现饥饿现象，为什么？

（2）使用 nice、cpuTime 和 waitTime 设计一种动态优先数计算方法，以避免产生饥饿现象，并说明 waitTime 的作用。

（1）静态优先级。若 $\text{priority} = \text{nice}$ ，一直有更小 nice 的就绪进程，新来的/低优先级者始终得不到 CPU，便可能饥饿；可抢占调度也不能自动保证它执行。

（2）动态式。例如 $\text{priority} = \text{nice} + \text{cpuTime} - \text{waitTime}$ （越小越优先）。运行者 cpuTime 逐渐增加，优先数值变大；就绪而未运行者 waitTime 逐渐增加，优先数值变小，久等者终会与对手竞争。waitTime 是老化项，必须用减号而非加号；题设在运行时将 waitTime 置 0、在就绪时将 cpuTime 置 0，防止把很久以前的历史无限叠加。

用两个进程检验符号

设 A 连续运行、B 一直就绪。过 k 拍后 A 的优先数值相对升 k，B 相对降 k；两者差每拍缩小 2，有限初始 nice 差终会被克服。若把 waitTime 也加上，只会让久等者越来越不易被选中。

02 | 2023-46：字符到来不等于进程马上返回

46. (8 分) 进程 P 通过执行系统调用从键盘接收一个字符的输入。已知此过程中与进程 P 相关的操作包括：① 将进程 P 插入就绪队列；② 将进程 P 插入阻塞队列；③ 将字符从键盘控制器读入系统缓冲区；④ 启动键盘中断处理程序；⑤ 进程 P 从系统调用返回；⑥ 用户在键盘上输入字符。以上编号 ①～⑥ 仅用于标记操作，与操作的先后顺序无关。请回答下列问题。

- （1）按照正确的操作顺序，操作 ① 的前一个和后一个操作分别是上述操作中的哪一个？操作 ⑥ 的后一个操作是上述操作中的哪一个？
- （2）在上述哪个操作之后，CPU 一定从进程 P 切换到其他进程？在上述哪个操作之后 CPU 调度程序才能选中进程 P 执行？
- （3）完成上述哪个操作的代码属于键盘驱动程序？
- （4）键盘中断处理程序执行时，进程 P 处于什么状态？CPU 处于内核态还是用户态？

（1）事件序列。P 发出读字符系统调用，暂无输入而被插入阻塞队列②；用户随后按键⑥；键盘中断启动处理④；驱动把字符从控制器读入系统缓冲③；等待条件满足，P 移入就绪队列①；被调度继续执行读操作并由系统调用返回⑤。所以①的前一项是③、后一项是⑤；⑥之后是④。编号只是标签，不能按 1 到 6 排序。

（2）状态与调度。②之后 P 阻塞、不可能继续占用 CPU，CPU 必须切给其它可运行进程（或空闲进程）；①之后 P 才成为候选，调度程序可选它执行，但就绪不等于马上运行。

(3) 驱动。将字符从键盘控制器读入系统缓冲的③属于键盘驱动的工作；中断处理的入口④使驱动得到执行机会，不能把用户按键⑥或用户程序返回⑤算成驱动代码。

(4) 中断时。若输入刚到时 P 仍在等这个字符，它处于阻塞态；CPU 响应中断时进入内核态，中断处理后才使 P 就绪。把“CPU 在内核态”错说成“P 已在运行态”会混淆两个不同主体。

三道状态门

系统调用等待使运行→阻塞；中断搬数据后使阻塞→就绪；调度选中才使就绪→运行。图中①是第二道门，⑤位于第三道门之后。

03 | 2025-46：指针变量和它指向的对象不在一处

46. (本题 9 分)

某进程的虚拟地址空间如下图所示，阴影部分为未占用区域。



题 46 图

另现有 C 程序代码：

```
char *ptr;
void main()
{
    int length;
    ptr = (char*)malloc(size of(char));
    scanf("%s", ptr);
    length = strlen(ptr);
    printf("length=%d\n", length);
    free(ptr);
}
```

请回答下列问题，并说明理由。

(1) 上述程序执行时，PCB 位于哪个区域，执行 scanf() 等待键盘输入时，该进程处于什么状态？(3 分)

(2) main() 函数的代码位于哪个区域？其直接调用的哪些函数的功能需要通过执行驱动程序实现？(3 分)

(3) 变量 ptr 被分配在哪个区域？若变量 length 没有被分配在寄存器中，则会被分配在哪个区域？ptr 指向的字符串位于哪个区域？(3 分)

- (1) 进程控制块 PCB 是操作系统管理的对象，在内核区；scanf 等键盘输入等待时进程是阻塞态。
- (2) main 的机器代码在只读代码区。直接调用的 scanf 要读取键盘，printf 要输出到终端，其 I/O 功能最终通过相应设备驱动程序完成；strlen 只是字符串计算，malloc/free 管理动态内存，不因这些调用本身就必然启动设备驱动。
- (3) 全局 char *ptr 这个指针变量本身在可读写数据区；未分配到寄存器的局部 length 在用户栈；malloc 返回指向的字符存储在运行时堆。原码只 malloc(sizeof(char)) 分配1字节，却用 %s 无长度限制读字符串，连非空字符串的结尾 \0 都容不下，实际执行有越界风险；这些区域判断只说明程序意图，不能保证这段输入代码安全执行。

一眼定位三种东西

看声明位置决定 ptr 变量存放处；看 malloc 决定它指向的对象存放处；看 main 的局部声明决定 length。同一行 ptr=malloc(...) 并不会把 ptr 变量搬到堆里。

复做顺序

先遮住答案，仅写01式子的正负号并用两进程验算；再默写02事件序列和三个状态门；最后在03逐个指出“变量/对象/代码”所在区域。尚需限时复做才可验证熟练度。